

String class in Java

25 August 2026 20:35

In Java, String is basically an object that represents sequence of char values. An array of characters works as a string in Java, for example:

```
char c[] = {'J', 'a', 'v', 'a'};
c
<object reference> (char[])
String s = new String(c);
s
"Java" (String)
```

Same as:

```
String s1 = "Java";
s1
"Java" (String)
```

Java String class provides a lot of methods to perform operations on String such as `compareTo()`, `concat()`, `equals()`, `split()`, `length()`, `substring()`, etc.

Generally, a **String** is a sequence of characters. However, a String in Java is an object that represents a sequence of characters. And the `java.lang.String` class is used to create String objects. Java strings are *immutable*. That means their value cannot be changed once created.

There are two ways to create string object:

- String literal
- Using new keyword

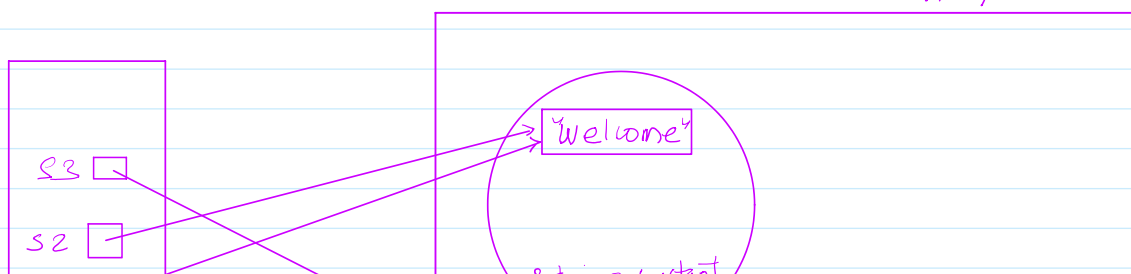
Creating string using string literal

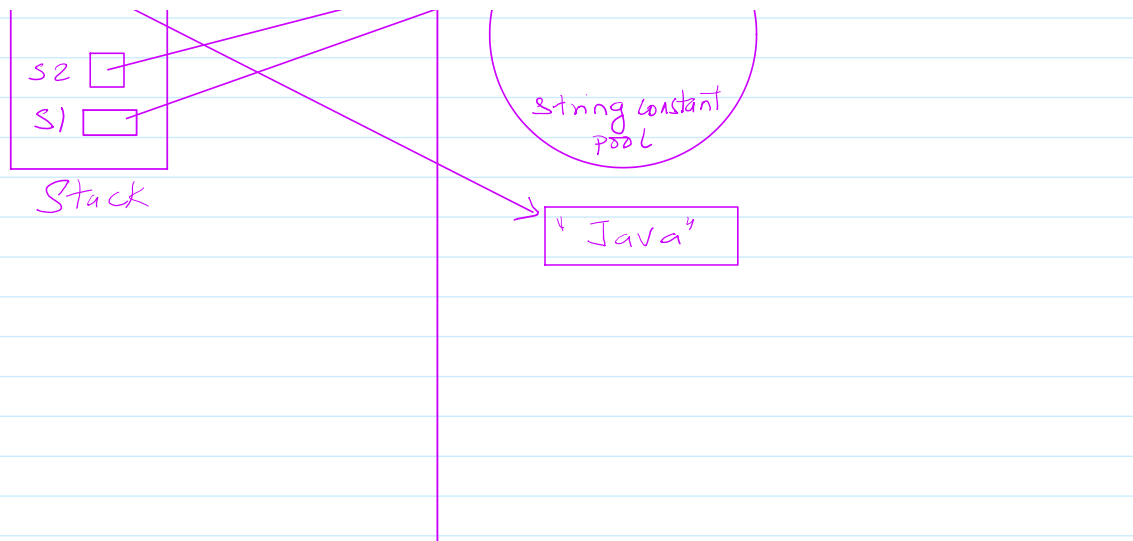
Java String literal is created by using double quotes. Example:

```
String s1 = "Java";
```

Each time you create a string literal, the JVM checks the string constant pool first. If the string already exist in the pool, a reference to the pooled instance is returned. If the string doesn't exist in the pool, a new String instance is created and placed in the pool. For example:

```
→String s1 = "Welcome";
→String s2 = "Welcome"; // it doesn't create a new instance.
```





In the above example, only one object will be created. Firstly, JVM will not find any string object with the value welcome in the String constant pool. That is why it will create a new object. After that it will find the string with the value welcome in the pool. It will not create a new object but will return the reference to the same instance.

2. Creating string using new keyword

A string can be created using the new keyword, which explicitly creates a new String object in memory rather than using the string literal pool.

For example:

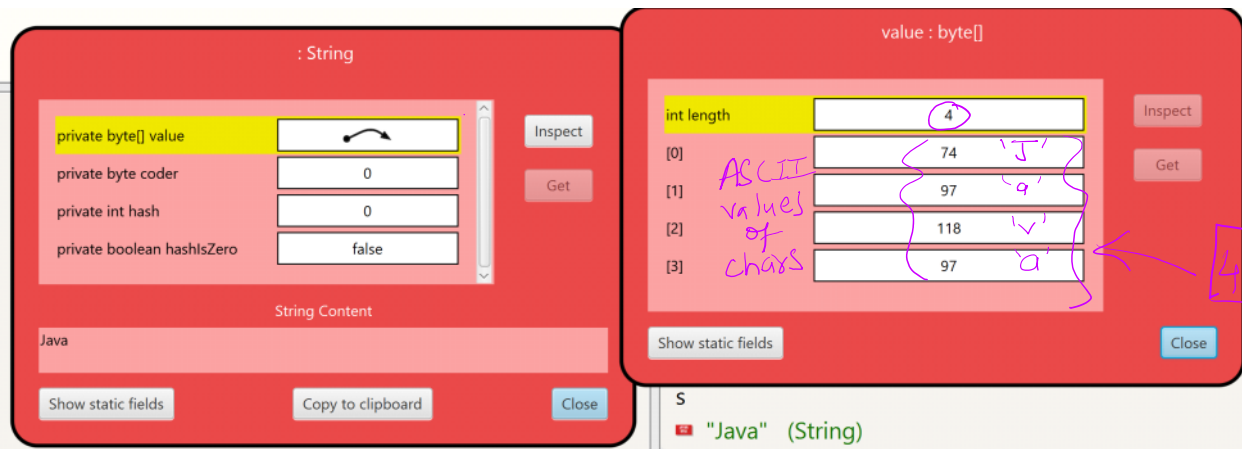
```
String s3 = new String("Java");
```

Here,

- "Java" Is a string literal, so it is stored in the string pool (one object).
- "new String("Java") Create a new String object in the heap (second object)
- s3 Is the reference variable that points to the object in the heap.

In such case JVM will create a new String object in normal (non-pool) heap memory and the literal "Welcome" will be placed in the String constant pool. The variable s3 will refer to the object in a heap (non-pool).

```
String s = new String("Java");
```



$\text{char}[] c = \{ 'J', 'a', 'v', 'a' \};$
 ↑
 2 bytes for each cell. $2 \times 4 = 8$ bytes occupied

Finding String Length

You can find the number of characters in a string by calling the `length ()` method on the string object.

```

S
"Java" (String)
int len = s.length();
len
4 (int)
  
```

Finding a Character in a String

You can find a character or substring in a string by using the `indexOf` method and return its position.

```
int index = s.indexOf("a");
```

$s \rightarrow$

0	1	2	3
J	a	v	a

```

index
1 (int)
  
```

Returning each character from the string.

```
char c = s.charAt(index);
```

```

package MyStringCodes;
import java.util.*;
  
```

```

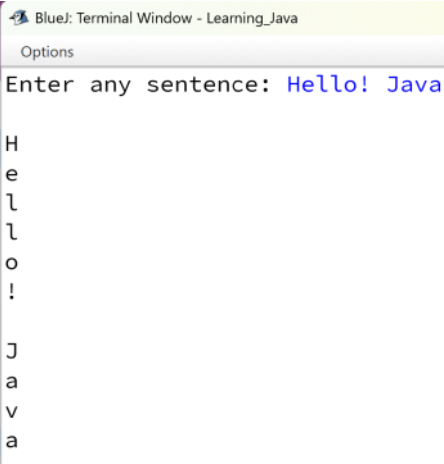
/**
 * Write a description of class StringDemo1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
  
```

```

public class StringDemo1
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter any sentence: ");
        String str = sc.nextLine();
        int i;
        for(i = 0; i < str.length(); i++){
            char c = str.charAt(i);
            System.out.print("\n" + c);
        }
    }
}

```

Output:



```

Blue: Terminal Window - Learning_Java
Options
Enter any sentence: Hello! Java

H
e
l
l
o
!
J
a
v
a

```

```

package MyStringCodes;
import java.util.*;

/**
 * Write a description of class NumberOfConsonantsAndVowels here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class NumberOfConsonantsAndVowels
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter any sentence: ");
        String str = sc.nextLine();
        int i, v = 0, conso = 0;
        String vowels = "AEIOUaeiou";
        String la="a", ua = "A";
        for(i = 1; i < 26; i++){
            //concatenation means joining of two strings.
            la += (char)(la.charAt(i-1) + 1); //explicit type casting
            ua += (char)(ua.charAt(i-1) + 1); //explicit type casting
        }
        System.out.print("\n" + la + "\n" + ua + "\n");
    }
}

```

```

        ua += (char)(ua.charAt(i-1) + 1); //explicit type casting
    }
    System.out.print("\n" + la + "\n" + ua + "\n");
    for(i = 0; i < str.length(); i++){
        char c = str.charAt(i);
        if((la.indexOf(c) != -1 || ua.indexOf(c) != -1)){
            if(vowels.indexOf(c) > -1){
                v++;
            }
            else{
                conso++;
            }
        }
    }
    System.out.print("\nVowels count: " + v);
    System.out.print("\nC consonant count: " + conso);
}
}

```

This function searches the value passed in the parameter, in the given string, if found, returns the index of first occurrence, otherwise returns '-1'.

Output:

```

BlueJ: Terminal Window - Learning_Java
Options
Enter any sentence: Welcome! Java

abcdefghijklmnopqrstuvwxyz
ABCDEFGHIJKLMNOPQRSTUVWXYZ

Vowels count: 5
C consonant count: 6

```

length()

charAt()

indexOf()